

Raport: Planowanie STRIPS - Blocks World

Autorzy: Patrick Bajorski, Jan Banasik **Data:** 2026-03-31

1. Wstęp

1.1 Cel projektu

Celem projektu jest implementacja i analiza algorytmu planowania STRIPS (Stanford Research Institute Problem Solver) na przykładzie problemu Blocks World. Projekt obejmuje:

- Zdefiniowanie problemów planistycznych w formalizmie STRIPS
- Implementację heurystyki przyspieszającej przeszukiwanie
- Porównanie efektywności algorytmu z heurystyką i bez niej
- Analizę dekompozycji celów na podcele (subgoals)

1.2 Blocks World

Blocks World to klasyczny problem planistyczny, gdzie:

- Mamy zbiór klocków, które można układać w stosy
- Klocki można przenosić tylko gdy są "wolne" (nic na nich nie leży)
- Można przenosić klocek na inny wolny klocek lub na stół
- Stół ma nieograniczoną pojemność

Reprezentacja stanu:

- $on(X) = Y$ - klocek X leży na Y (Y może być innym klockiem lub "table")
- $clear(X) = True/False$ - czy klocek X jest wolny (nic na nim nie leży)

Akcje:

- $move_X_from_Y_to_Z$ - przenieś klocek X z Y na Z
 - Warunki: $on(X) = Y, clear(X) = True, clear(Z) = True$ (jeśli Z nie jest stołem)
 - Efekty: $on(X) = Z, clear(Y) = True, clear(Z) = False$ (jeśli Z nie jest stołem)
-

2. Heurystyka Goal Mismatch

2.1 Definicja

Heurystyka **goal mismatch** (niedopasowanie celu) liczy ile warunków celu nie jest jeszcze spełnionych w bieżącym stanie:

$$h(state, goal) = |\{(feature, value) \in goal : state[feature] \neq value\}|$$

W implementacji:

```
def goal_mismatch_heur(state: StateAssignment, goal: Goal) -> float:
    return float(sum(1 for feat, val in goal.items() if state.get(feat) != val))
```

2.2 Uzasadnienie dopuszczalności

Heurystyka jest **dopuszczalna** (admissible) w naszym ustawieniu dla celów opisanych wyłącznie przez warunki typu $on(X) = Y$:

1. Cel składa się z przypisań $on(X)$ dla wybranych klocków.
2. Pojedyncza akcja $move_X_from_Y_to_Z$ zmienia tylko jedną zmienną typu $on(\cdot)$: dokładnie $on(X)$ dla przenoszonego klocka.
3. Zatem jeden ruch może „naprawić” co najwyżej jeden niespełniony warunek $on(X) \rightarrow$ liczba niespełnionych warunków jest dolnym ograniczeniem liczby ruchów.

2.3 Wpływ na przeszukiwanie A*

Algorytm A* z dopuszczalną heurystyką gwarantuje znalezienie optymalnego rozwiązania. Heurystyka goal mismatch:

- **Przyspiesza** przeszukiwanie, kierując je ku stanom bliższym celowi
- **Redukuje** liczbę rozwijanych węzłów w porównaniu do przeszukiwania bez heurystyki ($h=0$)
- Jest prosta obliczeniowo ($O(|goal|)$ na ewaluację)

3. Opis problemów testowych

3.1 Problemy małe (5 klocków) - wymagania 4-6 punktów

Problem	Stan początkowy	Cel	Opis
problem1	c na a, e na d, b sam	wieża a→b→c, d,e na stole	Budowa 3-wieży ze zblokowanych klocków
problem2	wieża a→b→c→d→e	dwie 2-wieże: b→a i d→c, e na stole	Rozkład wieży i budowa dwóch mniejszych
problem3	d na b, c na a, e sam	4-wieża e→d→c→b, a na stole	Budowa wysokiej wieży z zablokowaną podstawą

3.2 Problemy duże (12 klocków) - wymagania 8 punktów

Problem	Stan początkowy	Cel	Opis
problem4	Dwie 6-wieże: a→b→c→d→e→f i g→h→i→j→k→l	Dwie odwrócone 6-wieże: f→e→d→c→b→a i l→k→j→i→h→g	Odwrócenie dwóch wież

Problem	Stan początkowy	Cel	Opis
problem5	Dwie 6-wieże: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f$ i $g \rightarrow h \rightarrow i \rightarrow j \rightarrow k \rightarrow l$	Jedna 12-wieża: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f \rightarrow g \rightarrow h \rightarrow i \rightarrow j \rightarrow k \rightarrow l$	Połączenie wież
problem6	Jedna 12-wieża: $a \rightarrow b \rightarrow c \rightarrow \dots \rightarrow l$	Trzy 4-wieże: $d \rightarrow c \rightarrow b \rightarrow a$, $h \rightarrow g \rightarrow f \rightarrow e$, $l \rightarrow k \rightarrow j \rightarrow i$	Rozkład i przebudowa wieży

4. Wyniki eksperymentów

4.1 Problemy małe - tryb standardowy (4 punkty)

Heurystyka mismatch

Problem	Stany osiągalne	Rozwiązany	Koszt (akcje)	Węzły rozwinięte	Czas [s]
problem1	522	Tak	4	9	0.0009
problem2	555	Tak	4	5	0.0004
problem3	522	Tak	4	14	0.0014

Bez heurystyki (zero)

Problem	Stany osiągalne	Rozwiązany	Koszt (akcje)	Węzły rozwinięte	Czas [s]
problem1	522	Tak	4	319	0.0340
problem2	555	Tak	4	18	0.0016
problem3	522	Tak	4	199	0.0564

Porównanie - redukcja węzłów dzięki heurystyce

Problem	Węzły (zero)	Węzły (mismatch)	Redukcja
problem1	319	9	97.2%
problem2	18	5	72.2%
problem3	199	14	93.0%

Średnia redukcja: 87.5%

4.2 Problemy małe - tryb z subgoals (6 punktów)

Heurystyka mismatch + subgoals

Problem	Liczba subgoals	Koszt całkowity	Węzły rozwinięte	Czas [s]
problem1	3	4	10	0.0011

Problem	Liczba subgoals	Koszt całkowity	Węzły rozwinięte	Czas [s]
problem2	3	6	9	0.0010
problem3	3	4	7	0.0007

Bez heurystyki (zero) + subgoals

Problem	Liczba subgoals	Koszt całkowity	Węzły rozwinięte	Czas [s]
problem1	3	6	142	0.0164
problem2	3	6	49	0.0052
problem3	3	4	95	0.0124

Rozbicie na subgoals (przykład dla problem1 z heurystyką mismatch):

Subgoal	Opis	Koszt	Węzły	Akcje
1	Uwolnienie klocków a i d	2	3	move_c_from_a_to_table, move_e_from_d_to_table
2	Budowa wieży (b na c, a na b)	2	6	move_b_from_table_to_c, move_a_from_table_to_b
3	Cel końcowy (już spełniony po subgoal 2)	0	1	—

Rozbicie na subgoals (przykład dla problem3 z heurystyką mismatch):

Subgoal	Opis	Koszt	Węzły	Akcje
1	Odblokowanie klocka b	1	2	move_d_from_b_to_table
2	Budowa dolnej części wieży	2	4	move_c_from_a_to_b, move_d_from_table_to_c
3	Dokończenie wieży (e na d)	1	2	move_e_from_table_to_d

Porównanie: standardowy vs subgoals (heurystyka mismatch)

Problem	Węzły (standard)	Węzły (subgoals)	Koszt (standard)	Koszt (subgoals)
problem1	9	10	4	4
problem2	5	9	4	6
problem3	14	7	4	4

4.3 Problemy duże - tryb z subgoals (8 punktów)

Heurystyka mismatch + subgoals

Problem	Liczba subgoals	Koszt całkowity	Węzły rozwinięte	Czas [s]
problem4	3	20	42	0.078
problem5	3	21	91	0.156
problem6	3	20	39	0.057

Bez heurystyki (zero) + subgoals

Problem	Liczba subgoals	Koszt całkowity	Węzły rozwinięte	Czas [s]	Status
problem4	3	-	182794	303.4	TIMEOUT
problem5	3	-	194798	303.9	TIMEOUT
problem6	3	-	183747	303.8	TIMEOUT

5. Rozwiązania, wyniki i wizualizacje

Znalezione rozwiązania wraz z wizualizacją kroków rozwiązań oraz dodatkowymi szczegółami znajdują się w katalogu [blocksworld5/outputs/](#) w załączonym pliku **.zip**:

- [outputs/problemX/mismatch/](#) - rozwiązania z heurystyką
- [outputs/problemX/zero/](#) - rozwiązania bez heurystyki
- [outputs/problemX/subgoals_mismatch/](#) - rozwiązania z subgoals i heurystyką
- [outputs/problemX/subgoals_zero/](#) - rozwiązania z subgoals bez heurystyki

Każdy katalog zawiera:

- [step_XXX.png](#) - wizualizacja stanu po każdej akcji
- [solution_path.pdf](#) - wszystkie kroki w jednym PDF
- [results.json](#) - szczegółowe wyniki (koszt, czas, plan)

6. Analiza i wnioski

6.1 Wpływ heurystyki na efektywność

Obserwacje:

- Heurystyka mismatch zredukowała liczbę rozwijanych węzłów o średnio **87.5%** dla małych problemów
- Największa redukcja wystąpiła dla problemu 1 (**97.2%**) - z 319 do 9 węzłów
- Czas wykonania zmniejszył się średnio ok. **27-krotnie** (zależne od maszyny)
- Dla dużych problemów heurystyka jest **niezbędna** - bez niej solver nie znajduje rozwiązania w limicie 300s

Wnioski:

- Heurystyka goal mismatch jest bardzo efektywna mimo swojej prostoty
- Dla problemów z większą przestrzenią stanów korzyść z heurystyki rośnie eksponencjalnie
- Dopuszczalność heurystyki gwarantuje optymalność znalezionej rozwiązania

6.2 Dekompozycja na subgoals

Obserwacje:

- Podział na subgoals **zmniejszył** liczbę rozwijanych węzłów dla problem3 (z 14 do 7)
- Koszt rozwiązania z subgoals był **nieoptymalny** dla problem2 (6 vs 4 akcje)
- Dla problem1 i problem3 subgoals dały **optymalny** wynik (4 akcje)
- Dla dużych problemów subgoals były **niezbędne** do rozwiązania w rozsądnym czasie

Wnioski:

- Dekompozycja na subgoals może prowadzić do nieoptymalnych rozwiązań globalnie, nawet jeśli każdy podproblem jest rozwiązany optymalnie
- Jakość podziału ma kluczowe znaczenie - źle dobrane podcele mogą wymuszać niepotrzebne ruchy
- Dla złożonych problemów dekompozycja na subgoals jest kluczowa dla wydajności algorytmu poszukiwania rozwiązania

6.3 Skalowalność

Obserwacje:

- Dla 5 klocków przestrzeń stanów wynosi ~500 stanów osiągalnych (522-555)
- Dla 12 klocków przestrzeń stanów przekracza 10000 (limit pomiaru)
- Bez heurystyki rozwiązanie problemów dużych **nie było możliwe** w limicie 300s (timeout po ~180-195k węzłów w naszych testach)
- Z heurystyką mismatch + subgoals problemy duże rozwiązywane są w czasie **~0.06–0.16s** (39–91 węzłów)

Wnioski:

- Przestrzeń stanów rośnie wykładniczo z liczbą klocków
- Kombinacja heurystyki i dekompozycji na subgoals pozwala skalować algorytm do większych problemów
- Sama dekompozycja bez heurystyki jest niewystarczająca dla 12 klocków

6.4 Optymalność rozwiązań

Obserwacje:

- W trybie standardowym (cele opisane tylko przez $on(X)=Y$) A* z heurystyką mismatch znajduje rozwiązania optymalne (heurystyka jest wtedy dopuszczalna)
- Dekompozycja na subgoals **nie gwarantuje** optymalności globalnej
- Porównanie kosztów (tryb standardowy vs subgoals z heurystyką mismatch):
 - problem1: 4 vs 4
 - problem2: 4 vs 6 (subgoals o 50% gorsze)
 - problem3: 4 vs 4

Wnioski:

- Subgoals są kompromisem między optymalnością a możliwością rozwiązania trudniejszych problemów
 - Dobra definicja podceli powinna odpowiadać naturalnym etapom rozwiązania, nie wymuszać zbędnych ruchów
-

7. Podsumowanie

Projekt zrealizował następujące cele:

- Zdefiniowano 3 problemy małe (5 klocków) z ≥ 50 stanami osiągalnymi (522-555 stanów)
- Zdefiniowano 3 problemy duże (12 klocków) z rozwiązaniami ≥ 20 akcji
- Zaimplementowano heurystykę goal mismatch (dopuszczalna)
- Porównano efektywność z heurystyką i bez niej
- Zaimplementowano dekompozycję celów na subgoals (2 podcele + cel końcowy dla każdego problemu)
- Wygenerowano wizualizacje rozwiązań

Kluczowe wnioski:

1. Heurystyka goal mismatch redukuje liczbę rozwijanych węzłów o ~88-97% dla małych problemów i jest niezbędna dla dużych problemów
2. Dekompozycja na subgoals umożliwia rozwiązywanie problemów o dużej przestrzeni stanów kosztem potencjalnej utraty optymalności
3. Kombinacja heurystyki i subgoals pozwala skalować planowanie STRIPS do problemów z 12 klockami (rozwiązanie w ~0.03s vs timeout bez heurystyki)